

CSE 470

Software Engineering

Mid-Term Review

Imran Zahid

Lecturer

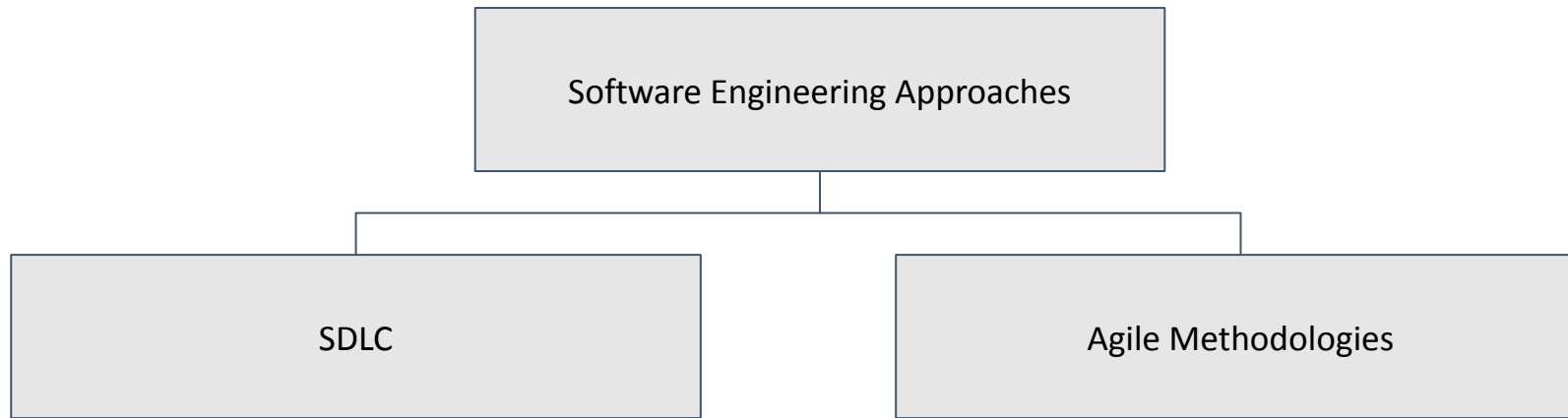
Computer Science and Engineering, BRAC University



Syllabus

- Software Process Models
 - Waterfall
 - V-Model
 - Incremental
 - Iterative
- Agile Development Process
 - Extreme Programming (XP)
 - Scrum
- Requirement Engineering
 - Functional Requirements
 - Non-Functional Requirements
- UML Design
 - Class Diagrams
- Software Architectural Patterns
 - Layered
 - Model-View-Controller (MVC)
 - Repository
 - Client-Server
 - Microservices

Software Engineering Approaches



- This topic includes both SDLC and Agile Methodologies
- Remember, Agile Methodologies are NOT SDLC. If the question specifies SDLC, the answer can NOT be Agile. If the question mentions software engineering approach, it can be both SDLC and Agile.

Software Engineering Approaches

	Waterfall	V Model	Incremental	Iterative	XP	Scrum
Small	✓	✓	✗	✗	✗	✗
Medium	✗	✓	✓	✓	✗	✓
Large	✗	✗	✓	✓	✓	✓
Testing	✗	✓	✓	✓	✓	✓
Initial Req. Changing	✗	✗	✗	✓	✓	✓
Process Req. Changing	✗	✗	✓	✓	✓	✓
Early Release	✗	✗	✓	✗	✓	✓
Technologies Changing	✗	✗	✗	✓	✗	✓

← Prioritize

Example

A software company has been hired by a startup to develop a mobile app for managing personal finances. The startup has an ambitious plan, but their **requirements are not fully defined**, as they are still exploring the exact features they want. They need to get a **working version of the app out to market quickly** and plan to gather feedback from early users to adjust the product based on their needs. The app's functionality will **likely evolve over time based on this user feedback**, and the startup expects **frequent changes and improvements**.

The development team must work closely with the startup, holding regular meetings to review progress and adjust the project's direction as needed. The team needs to prioritize flexibility, fast iteration, and continuous delivery of working versions of the app.



Example

A software company has been hired by a startup to develop a mobile app for managing personal finances. The startup has an ambitious plan, but their **requirements are not fully defined**, as they are still exploring the exact features they want. They need to get a **working version of the app out to market quickly** and plan to gather feedback from early users to adjust the product based on their needs. The app's functionality will **likely evolve over time based on this user feedback**, and the startup expects **frequent changes and improvements**.

The development team must work closely with the startup, holding regular meetings to review progress and adjust the project's direction as needed. The team needs to prioritize flexibility, fast iteration, and continuous delivery of working versions of the app.

Answer: Agile Methodology (Changing initial requirements + Early release)

Specifically, Scrum (Fast iteration [1-4 weeks] + Regular meetings)



Example

You are assigned to develop a **medium-sized** software project for a client who is **well-aware of their requirements** and wants to start using a **prototype of the software from the very beginning**. The client also wishes to prioritize features in each delivery.



Example

You are assigned to develop a **medium-sized** software project for a client who is **well-aware of their requirements** and wants to start using a **prototype of the software from the very beginning**. The client also wishes to prioritize features in each delivery.

Answer: Incremental (Clear requirements + Early release)



Example

You are managing a **small-scale** software project for FinTrust Ltd., a financial institution aiming to develop an auditing application named AuditGuard. The client, represented by Mr. Thomas, has **clearly defined and stable requirements**, which include **generating monthly, quarterly, and annual audit reports, user access management, secure data storage, detailed audit trails, and integration with existing accounting software.**

Mr. Thomas has stressed the importance of **reliability and accuracy**, expecting the software to meet stringent financial **compliance standards**. Additionally, he insists on **meticulous documentation** at each step to ensure clarity and facilitate compliance audits.



Example

You are managing a **small-scale** software project for FinTrust Ltd., a financial institution aiming to develop an auditing application named AuditGuard. The client, represented by Mr. Thomas, has **clearly defined and stable requirements**, which include **generating monthly, quarterly, and annual audit reports, user access management, secure data storage, detailed audit trails, and integration with existing accounting software.**

Mr. Thomas has stressed the importance of **reliability and accuracy**, expecting the software to meet stringent financial **compliance standards**. Additionally, he insists on **meticulous documentation** at each step to ensure clarity and facilitate compliance audits.

Answer: V-Model (Clear requirements, small-scale, needs testing and documentation, doesn't need early release)



Software Engineering Approaches - Questions

- Why did you choose your SE Approach? Justify
 - Write the pros and cons/advantages and disadvantages of your chosen SE Approach (optionally, in the context of the scenario).
 - Why not choose other SE Approaches that might also be used in this scenario? (particularly talking about approaches that also apply, in this case you have to explain the overhead of other approaches)
 - Why not choose other SE approaches? (explain the cons of the other approaches that made you not choose them)



Software Engineering Approaches - Questions

- Describe the process of your chosen SE Approach (diagram or textual description. In case of Agile, Waterfall and V-Model, break down the individual steps. In case of XP, mention the core principles)
- Explain the importance of a particular individual step in the process of your chosen SE Approach.
- Apply the process of your chosen SE Approach to the given scenario (break down the individual steps of your process and relate them to the scenario)



Requirements Engineering

- Requirements
 - Functional Requirements
 - Non-Functional Requirements
- Core question: Identify User and System Requirements/Functional and Non-functional Requirements from a given scenario.



Requirements Engineering

- Functional Requirements might be User or System requirements, try to identify all actions done by an user using the system, or actions done by the system.
- Non-functional Requirements might not always be explicitly stated, assume some based on the context. E.g. an insulin pump should not have downtime longer than the duration between doses. Non-functional Requirements must have some metric.
- Each Functional Requirement should be an individual task.



Example

In the country Romania, where agriculture is a way of life, a modern solution has emerged to transform traditional farming practices into efficient and productive endeavors. The Agriculture Management System has become an important part in the lives of farmers, government agencies, and suppliers alike. There are various stakeholders such as government officers, marketers/suppliers, and farmers. Farmers are central to the system. They use the system to manage their crops, land parcels, and harvests efficiently. Farmers can add, view, and update information about their crops. As crops mature, their quantity and harvest dates are tracked. Farmers own multiple land areas for cultivation, and the system helps them manage the details of area and soil type of the land areas. There are many markets and different types of products in each market. Government agencies, led by individuals like Officer Rahman, utilize the system to generate and manage Reports. These reports contain details such as crop production rate and sale rate. Suppliers, like Samir, use the system to coordinate orders.



Functional Requirements - Farmers

- Allow farmers to add/view/update information about their crops.
- Track the maturation of crops.
- Monitor the quantity of crops as they mature.
- Keep records of harvest dates for crops.
- Help farmers manage multiple land areas for cultivation.
- Provide details on the area of each land parcel.
- Offer information on the soil type of each land parcel.



Functional Requirements - Suppliers

- Store data about many markets.
- Provide information on different types of products in each market.
- Allow suppliers to coordinate orders with farmers.



Functional Requirements - Officers

- Enable government officers to generate reports with details such as crop production rate and sale rate.
- Allow government officers to manage reports



Non-Functional Requirements

- Reliably track crop maturation, quantities, and harvest dates, the predicted harvest date should not differ from the actual date of crop maturation by more than **2 days**.
- Handle many types of crops and multiple land areas without performance degradation, the system should handle upto **50 different crops** with less than 5% performance degradation.
- Comply with **government regulations and standards** relevant to agriculture in **Rovinia**.



Class Diagrams

- Core Question: Draw a class diagram for a given scenario.



Example

A bookstore system manages books and authors. Books have titles, publication dates, and prices. Authors write books and have names and biographies. Customers can purchase books. Each purchase generates a receipt.



Nouns (Potential Classes)

- Book
- Author
- Title
- Publication Date
- Price
- Customer
- Purchase
- Receipt
- Biography



Verbs (Potential Methods)

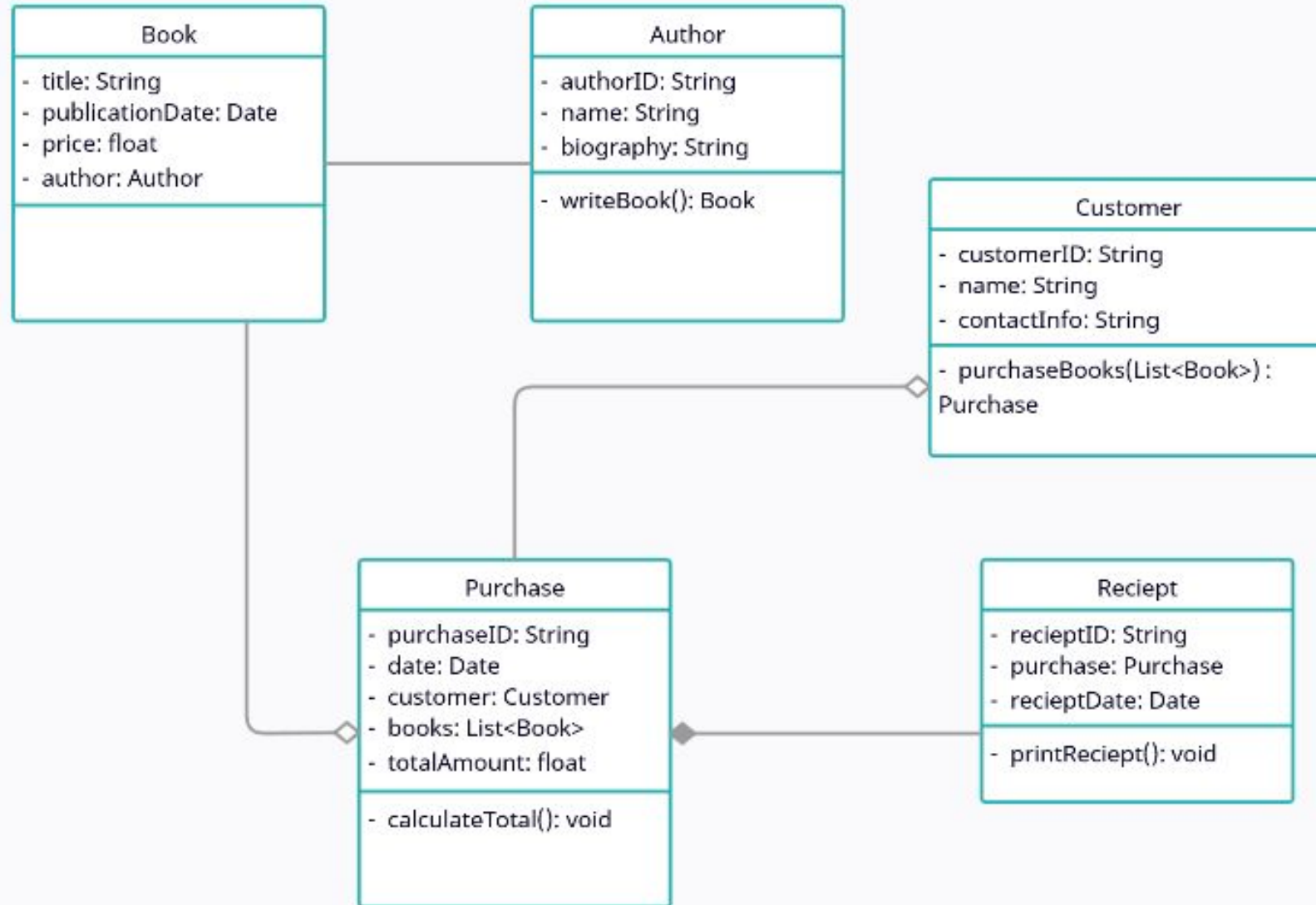
- Manages books and authors (The bookstore system manages books and authors)
- Have titles, publication dates, and prices (Books have titles, publication dates, and prices)
- Write books (Authors write books)
- Have names and biographies (Authors have names and biographies)
- Can purchase books (Customers can purchase books)
- Generates a receipt (Each purchase generates a receipt)



Prepositions, Conjunctions, Verbs between Objects (Potential Relationships)

- Have (Books have titles, publication dates, and prices.)
- Generates (Each purchase generates a receipt)
- Writes (Authors write books)
- Manages (A bookstore system manages books and authors.)
- Purchase (Customers can purchase books)





Software Architecture

- Core Questions
 - Given a scenario, which Software Architecture would you choose to use?
 - Based on the choice, how does the Software Architecture meet the needs of the scenario / describe the components in the given scenario.
 - In case of Layered/MVC, describe what parts of the software would be contained in each component.
 - In case of Microservices/Repository/Client-Server the easiest way is to draw an architecture diagram with the components labeled as parts of the scenario.
 - Write the characteristics/describe the components of an architecture.



Software Architecture

- Primary Concern
 - I need to structure the system into well-defined functional parts (and doesn't focus on dealing with large amounts of data). → Layered or MVC
 - E.g. need reusability or maintainability
 - I need to manage and share large volumes of data, or provide a variety of services on some data → Repository or Client-Server or Microservices



Layered or MVC

- If the system focuses on UI flexibility and separation between frontend and backend → Choose MVC
 - That is, it is a general purpose system with no additional constraints
- If the system primarily focuses on separating internal logic into clear layers → Choose Layered
 - It is a general purpose system, but with additional constraints such as the need for separate security layers (e.g. multilevel security), a need for a separate services layer, etc.



Client-Server or Repository

- If multiple (potentially unlimited) users need remote access to shared data → Choose Client-Server
 - The number of users/subsystems needs to be able to scale up to any amount (i.e. scalability is a concern)
- Same as Client-Server, but need more scalability, and flexibility -> Choose Microservices
- If data consistency and centralized internal storage are the main concerns → Choose Repository
 - Need to store large amounts of data centrally for a (limited) number of subsystems, and perhaps need to perform some action on a central storage of data (e.g. backup, etc.)



Example

A government agency is developing a **nationwide healthcare management system** that will integrate multiple independent services, including **patient records, hospital inventory, billing, and insurance processing**. Each of these services is developed by separate teams, and they must function independently while still being part of a unified system.

Additionally, the system must enforce **multiple layers of security and access control**, ensuring that only authorized personnel can access specific types of data. The agency also requires the ability to introduce **a separate auditing and logging layer** to track all activities without modifying the core business logic.



Example

A government agency is developing a **nationwide healthcare management system** that will integrate multiple independent services, including **patient records, hospital inventory, billing, and insurance processing**. Each of these services is developed by separate teams, and they must function independently while still being part of a unified system.

Additionally, the system must enforce **multiple layers of security and access control**, ensuring that only authorized personnel can access specific types of data. The agency also requires the ability to introduce **a separate auditing and logging layer** to track all activities without modifying the core business logic.

Answer: Layered Architecture



Example

A startup is developing a modern e-commerce platform where customers can browse products, add them to their cart, and place orders. The company expects frequent changes to the user interface as they experiment with new layouts and interactive features. The system must allow multiple front-end developers to work on the user experience while back-end developers focus on business logic and data management. The same business logic should also support different interfaces, including a web application, a mobile app, and a chatbot integration.



Example

A startup is developing a modern e-commerce platform where customers can browse products, add them to their cart, and place orders. The company expects frequent changes to the user interface as they experiment with new layouts and interactive features. The system must allow multiple front-end developers to work on the user experience while back-end developers focus on business logic and data management. The same business logic should also support different interfaces, including a web application, a mobile app, and a chatbot integration.

Answer: MVC



Thank you

